PROJECT REPORT

EXPERIMENT - 04

Project title:

AIR POLLUTION MONITORING SYSTEM USING GAS SENSOR

Submitted by:

Arundeepak S – 24MER004
Deepak p – 24MER006
Dharanidharan R S – 24MER007
Dinesh K – 24MER009
Vishnivarthan P – 24MER063

IoT-Based Air Pollution Monitoring System

1. Project Overview

This project implements an IoT-based air pollution monitoring system that continuously measures gas concentration levels in the surrounding air and raises a local alert (buzzer + LED) whenever readings move outside a safe, configurable range. It is built as an early-warning module for industrial and environmental safety applications, where real-time gas data can be used to detect harmful gas leakage, prevent hazardous exposure, and support timely intervention before conditions become dangerous.

The prototype was built and debugged end-to-end on a NodeMCU (ESP8266) development board with an MQ2 gas sensor, a 16x2 I2C LCD display, a buzzer, and a status LED, with cloud connectivity added via the ThingSpeak IoT platform for remote data logging and visualization.

2. Problem Statement

Industrial and residential environments are frequently exposed to harmful gases such as LPG, smoke, carbon monoxide, and methane, often released due to leakage, incomplete combustion, or faulty appliances. Conventional monitoring relies on manual inspection or basic non-networked alarms, which do not provide continuous or remote visibility. This leads to:

- Delayed detection of gas leakage or poor air quality
- Risk of fire, explosion, or health hazards due to late response
- No remote visibility into air quality conditions for supervisors or residents

There is a need for a low-cost, easily deployable IoT sensing unit that can continuously monitor gas concentration, provide an immediate local alert when conditions go out of range, and optionally upload data to the cloud for remote monitoring and trend analysis.

3. Proposed Solution

A single-zone IoT air quality monitoring node built around a NodeMCU (ESP8266) microcontroller was proposed and implemented with the following capabilities:

- Continuous gas concentration sensing using an MQ2 sensor, sampled every 1 second
- Local threshold-based alerting: buzzer and LED trigger immediately when gas value goes below or above configured safe limits
- On-device display of live gas value and status on a 16x2 I2C LCD
- Serial diagnostics for real-time debugging and verification during development
- Cloud data logging to ThingSpeak over Wi-Fi, enabling remote visualization of gas trends and status via a lamp indicator

4. System Architecture

The system follows a layered IoT architecture:

- Sensing Layer: MQ2 sensor captures analog gas concentration in the surrounding air.
- Edge/Control Layer: NodeMCU reads the sensor, evaluates threshold logic, and drives the buzzer, LED, and LCD directly — this works even without network connectivity.
- Connectivity Layer: ESP8266 Wi-Fi radio connects to a local network/hotspot (2.4 GHz only).
- Cloud Layer: Sensor readings are pushed to a ThingSpeak channel via HTTP requests to the ThingSpeak Update API.
- Visualization Layer: ThingSpeak's built-in channel charts and lamp indicator widget display live gas value and status.

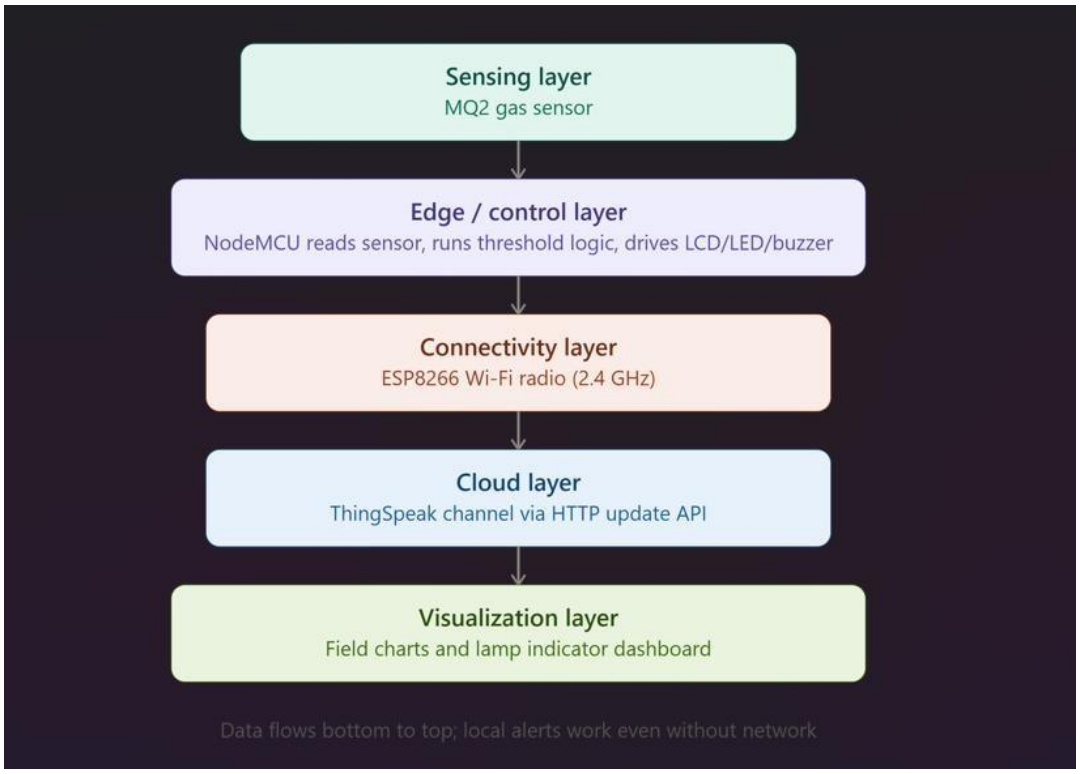

Fig. 1 Layered IoT system architecture

5. Circuit Table

Component	Pin	**NodeMCU**	GPIO
MQ2 Gas Sensor	VCC	3V3	–
MQ2 Gas Sensor	A0 (Analog Out)	A0	ADC0
MQ2 Gas Sensor	GND	G	–
I2C LCD (16x2)	VCC	3V3/VIN	–
I2C LCD (16x2)	SDA	D2	GPIO4
I2C LCD (16x2)	SCL	D1	GPIO5

Component	Pin	NodeMCU Pin	GPIO
I2C LCD (16x2)	GND	G	–
Buzzer	I/O (Signal)	D5	GPIO14
Buzzer	GND	G	–
Status LED	Anode (+) via 220Ω resistor	D6	GPIO12
Status LED	Cathode (−)	G	–
Component	Pin	NodeMCU Pin	GPIO
MQ2 Gas Sensor	VCC	3V3	–
MQ2 Gas Sensor	A0 (Analog Out)	A0	ADC0
MQ2 Gas Sensor	GND	G	–
I2C LCD (16x2)	VCC	3V3/VIN	–
I2C LCD (16x2)	SDA	D2	GPIO4
I2C LCD (16x2)	SCL	D1	GPIO5
I2C LCD (16x2)	GND	G	–
Buzzer	I/O (Signal)	D5	GPIO14
Component	Pin	NodeMCU Pin	GPIO

Buzzer	GND	G	–
Status LED (single)	Anode (+) via 220Ω resistor	D6	GPIO12
Status LED (single)	Cathode (−)	G	–

6. Circuit Design

The circuit is a single-board prototype centered on the NodeMCU ESP8266. Power is supplied via USB (5V), which is regulated on-board to 3.3V for the sensor logic, while the LCD backlight is powered from VIN (5V) for reliable brightness. Design considerations:

- MQ2 gives an analog voltage output proportional to gas concentration, read through the ESP8266's single analog pin (A0).
- The I2C LCD communicates over two lines (SDA, SCL), reducing wiring complexity compared to a parallel LCD.
- The buzzer and LED are digital-output driven; a 220Ω resistor is used with the LED to limit current.

The MQ2 sensor requires a warm-up/preheat period of 1–2 minutes after power-on before giving stable readings — this was factored into testing.

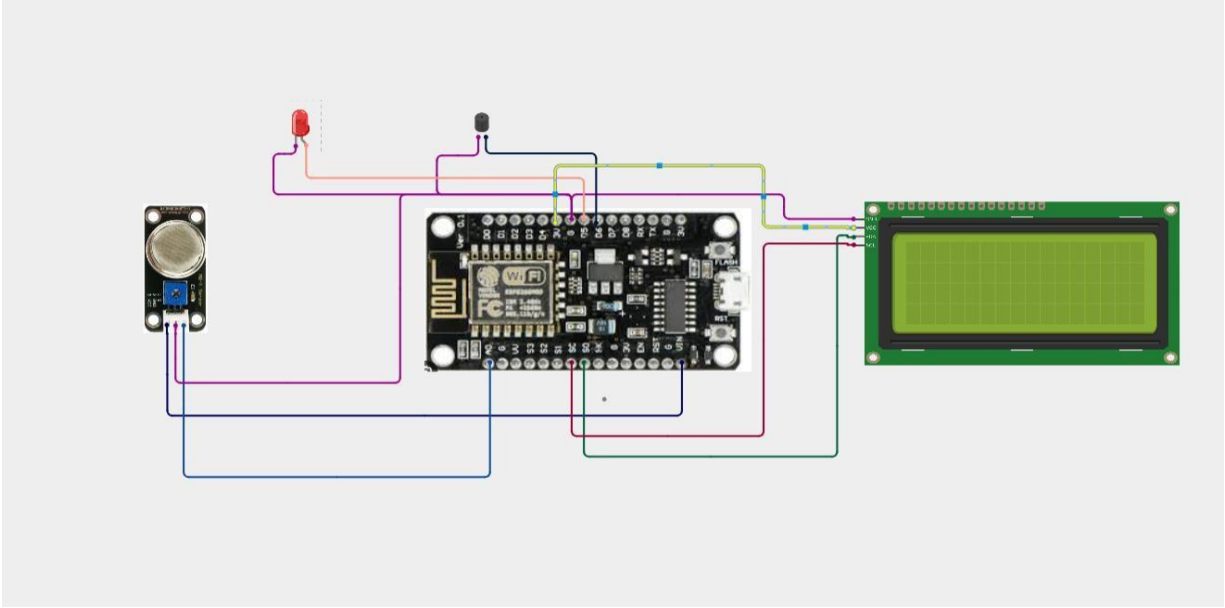

Fig. 2 Circuit wiring diagram

7. Algorithm

Main control loop logic (executes continuously once the board boots):

1. Initialize serial output, buzzer pin, LED pin, and I2C LCD.
2. Display a startup confirmation message ("LCD Test OK") on the LCD.
3. Connect to Wi-Fi and initialize the ThingSpeak client.
4. Every 1 second, read the analog gas value from the MQ2 sensor (A0).
5. Print the gas value to the Serial Monitor and display it on line 1 of the LCD.
6. Evaluate: statusBad = (gasValue < lowLimit) OR (gasValue > highLimit).
7. If statusBad is true, turn the LED ON, turn the buzzer ON, and display "Status: BAD" on line 2 of the LCD and Serial Monitor.
8. If statusBad is false, turn the LED OFF, turn the buzzer OFF, and display "Status: GOOD".
9. Send the gas value (Field 1) and status as 0/1 (Field 2) to the ThingSpeak channel via Wi-Fi.

8. Coding

The final working firmware (MQ2 + LCD + buzzer + LED alert + ThingSpeak upload):

cpp

```
#include <ESP8266WiFi.h>
#include <ThingSpeak.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);

const char* ssid = "YOUR_WIFI_NAME";
const char* password = "YOUR_WIFI_PASSWORD";
unsigned long channelID = 0;
const char* apiKey = "YOUR_WRITE_API_KEY";
WiFiClient client;
int mq2Pin = A0;
int buzzer = D5;
int greenLED = D6;

int lowLimit = 160;
int highLimit = 400;

void setup() {
  Serial.begin(115200);
  pinMode(buzzer, OUTPUT);
  pinMode(greenLED, OUTPUT);

  lcd.init();
```

```
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print("LCD Test OK");
  delay(2000);
  lcd.clear();

  WiFi.begin(ssid, password);
  Serial.print("Connecting to WiFi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWiFi Connected!");

  ThingSpeak.begin(client);
}

void loop() {
  int gasValue = analogRead(mq2Pin);
  int status = 0;

  Serial.print("Gas Value: ");
  Serial.println(gasValue);

  lcd.setCursor(0, 0);
  lcd.print("Gas: ");
  lcd.print(gasValue);
  lcd.print("    ");
```

```
  lcd.setCursor(0, 1);

  if (gasValue < lowLimit || gasValue > highLimit) {
    status = 1;
    digitalWrite(greenLED, HIGH);
    digitalWrite(buzzer, HIGH);
    lcd.print("Status: BAD    ");
    Serial.println("Status: BAD");
  } else {
    status = 0;
    digitalWrite(greenLED, LOW);
    digitalWrite(buzzer, LOW);
    lcd.print("Status: GOOD   ");
    Serial.println("Status: GOOD");
  }

  ThingSpeak.setField(1, gasValue);
  ThingSpeak.setField(2, status);
  int result = ThingSpeak.writeFields(channelID, apiKey);

  if (result == 200) {
    Serial.println("ThingSpeak update success!");
  } else {
    Serial.println("ThingSpeak update failed, code: " + String(result));
  }
  delay(20000);
}
```

9. System Working

On power-up, the NodeMCU initializes all peripherals and shows "LCD Test OK" on the display to confirm wiring. It then enters a continuous sensing loop:

- Every 1 second, the MQ2 sensor's analog value is read.
- Valid readings are printed to the Serial Monitor and shown on line 1 of the LCD (e.g., "Gas: 234").
- If the reading falls below 160 or exceeds 400, the system enters an alert state: the LED turns on and the buzzer sounds, with line 2 of the LCD and Serial Monitor showing "Status: BAD".
- The system automatically clears the alert once readings return to the safe range (160–400), displaying "Status: GOOD".
- The board connects to a Wi-Fi access point (2.4 GHz only) and pushes the latest gas value and status to ThingSpeak every 20 seconds, where it appears as live charts and a lamp indicator on the channel dashboard.

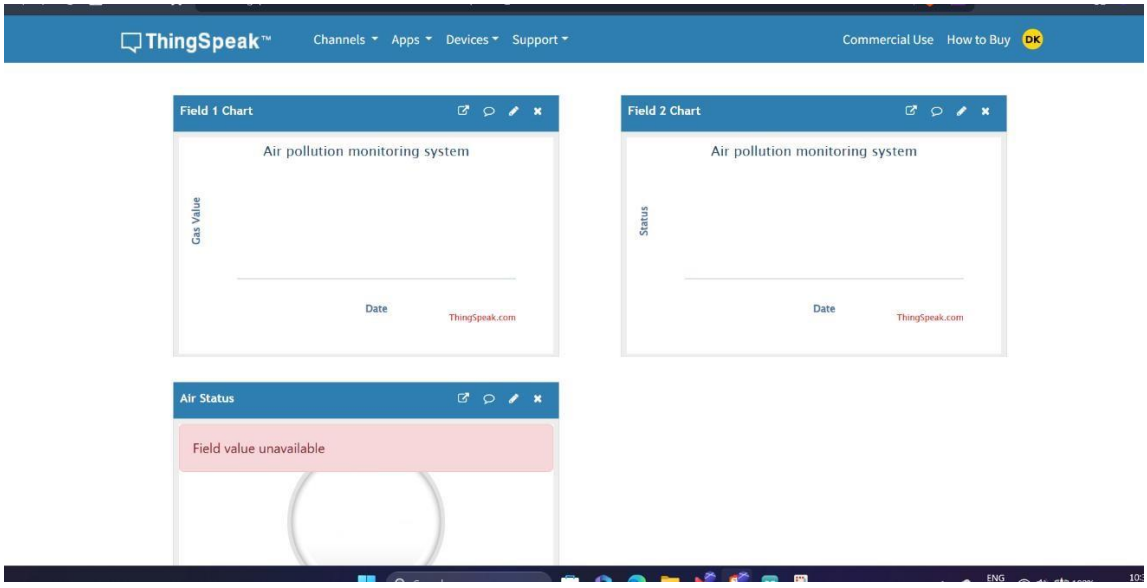

Fig. 3 ThingSpeak dashboard — Field 1 and Field 2 charts (initial/empty)

10. Bill of Materials

Component	Quantity	Purpose
NodeMCU ESP8266 (ESP-12E)	1	Main microcontroller with built-in Wi-Fi
MQ2 Gas Sensor	1	Harmful gas / smoke detection
OLED/LCD Display (16x2 I2C)	1	Local display of gas value and status
Buzzer	1	Audible alert
LED Indicators	2	Visual status indication
Breadboard	1	Prototyping platform
Jumper Wires	1 set	Connections
Micro-USB cable (data-capable)	1	Programming and power
USB power source (PC port or 5V/1A+ adapter)	1	Power supply

11. Prototype Implementation

The prototype was assembled and debugged iteratively, with the following notable steps and issues resolved during implementation:

- Initial upload attempts failed due to a Windows USB serial driver conflict; this was corrected after diagnosing via Windows Device Manager.
- Serial Monitor initially showed no output because the code only called Serial.begin() without any Serial.print() statements; this was corrected by adding print statements inside the loop.
- The first display attempt used SSD1306 OLED code on hardware that was actually a character-type I2C LCD (JHD 162A), causing a blank screen; this was identified by physically inspecting the display module and confirmed by testing.
- An I2C scanner sketch was used to identify the correct I2C address of the display module, which was found to be 0x3E for one module and 0x27 for a second replacement module — different from the commonly assumed default addresses.
- A display that appeared powered (module LED glowing) but showed no text was diagnosed as a contrast/wiring issue; the onboard potentiometer was checked and the backlight jumper was confirmed present.
- Only one LED was available instead of two, so the alert logic was redesigned to use a single LED (ON = danger, OFF = safe) combined with the buzzer, instead of separate red/green LEDs.
- The alert threshold logic was updated from a single upper threshold to a dual-band range (160–400), so both unusually low and high readings trigger a warning.
- Cloud integration required creating a ThingSpeak channel with two fields (gas value and numeric status), and configuring a lamp indicator widget on Field 2 to visually represent the BAD/GOOD state remotely.

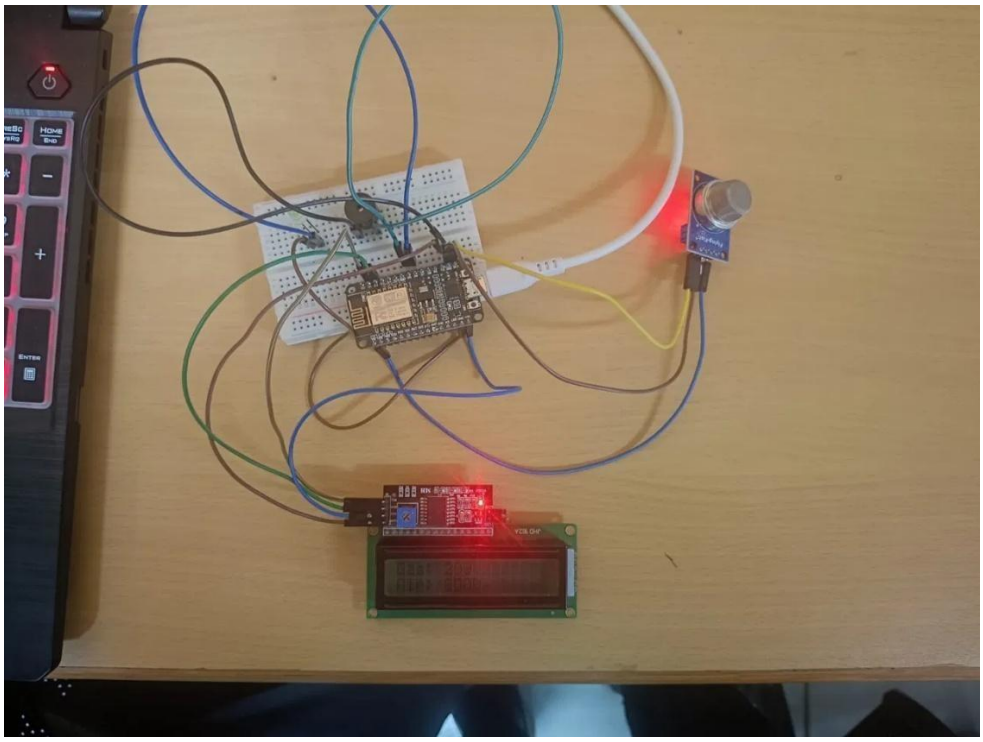

Fig. 4 Assembled prototype — MQ2 sensor, LCD, buzzer, LED wired to NodeMCU

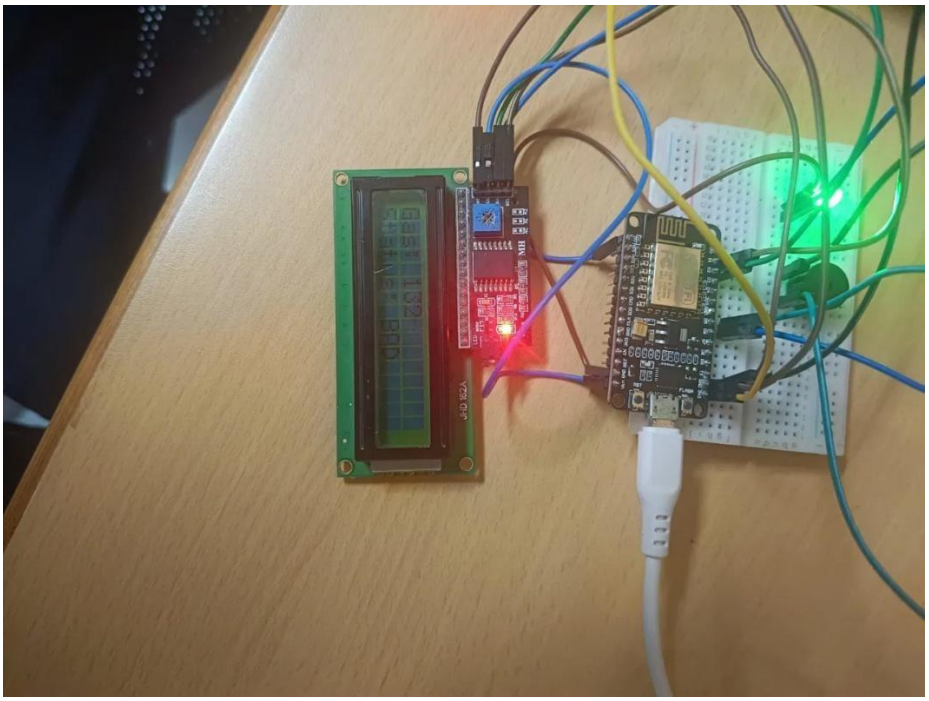

Fig. 5 Prototype — alternate/close-up view

12. Results & Performance Analysis

Once wiring and configuration issues were resolved, the system performed as designed:

- The MQ2 sensor produced consistent analog readings (baseline ~230–235) that responded visibly to disturbance, such as breath or nearby gas sources, confirming correct sensor operation.
- The LCD correctly displayed the live gas value on line 1 and the current status ("GOOD"/"BAD") on line 2, updating every second.
- The alert logic correctly triggered the LED and buzzer whenever the gas value moved outside the configured 160–400 safe range.
- Serial Monitor output matched the LCD display, confirming consistent status reporting across both interfaces.
- Cloud upload to ThingSpeak was implemented with a 20-second update interval, respecting the platform's 15-second minimum, with gas value logged to Field 1 and status (0/1) logged to Field 2, visualized using a lamp indicator widget.

Overall, the prototype demonstrates a working local sensing-and-alert pipeline with a clear path to full cloud-connected operation. Remaining work includes calibrating the MQ2 threshold values against known gas concentrations (e.g., using a lighter or alcohol swab) rather than relying on estimated baseline values, and extending the system to multiple sensing points for wider industrial coverage.

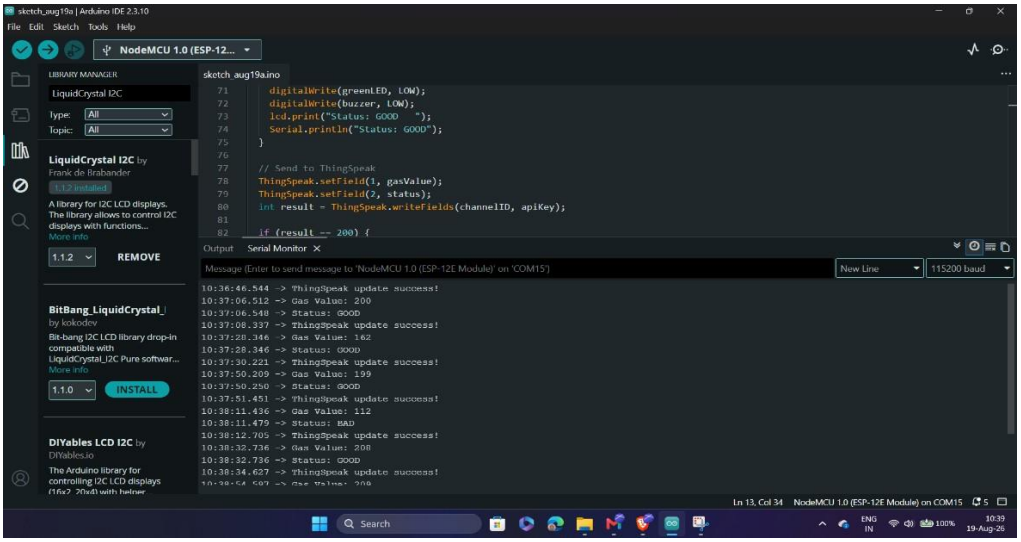

Fig. 6 Serial Monitor — live gas readings and ThingSpeak upload log

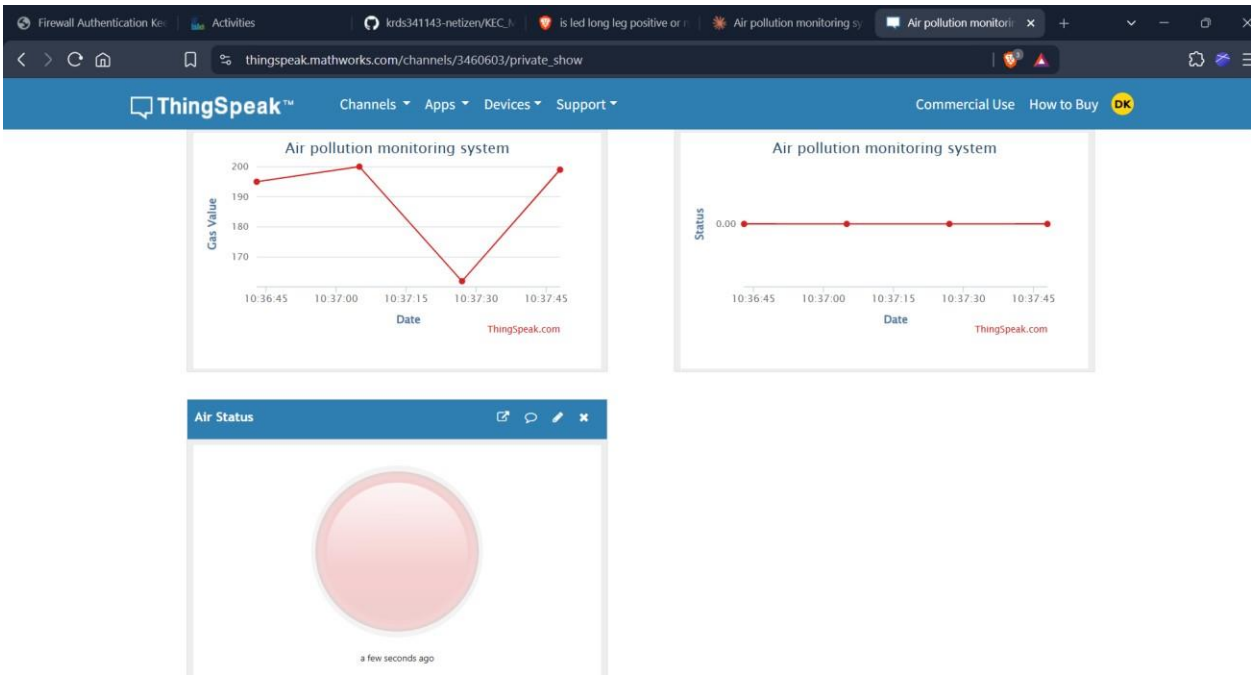

Fig.7 ThingSpeak dashboard — live data with GOOD status (lamp green/off)

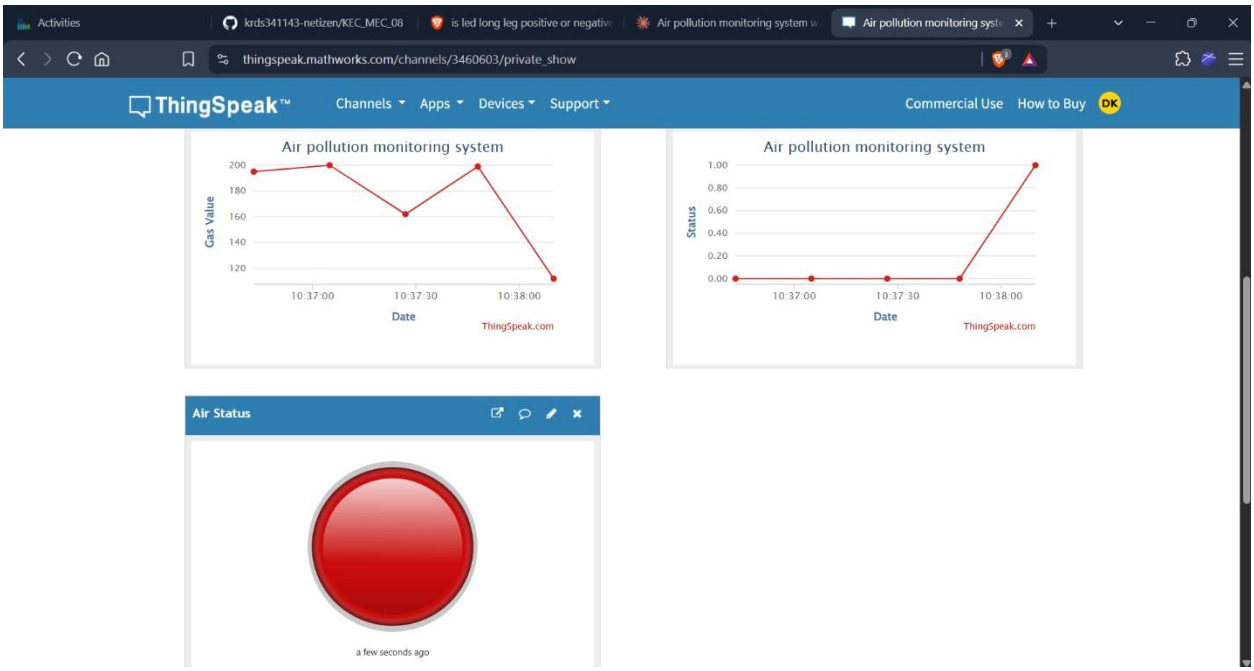

Fig. 8 ThingSpeak dashboard — live data with BAD status (lamp red/on)